

1. LIST

Doubly Linked List STL.

Header:

```
#include <list>
```

Declaration:

```
list<int> l;
```

Functions:

```
push_back()  
push_front()  
  
pop_back()  
pop_front()  
  
front()  
back()  
  
insert()  
erase()  
  
size()  
empty()  
  
clear()
```

Example:

```
list<int> l;  
  
l.push_back(10);  
l.push_front(5);  
  
for(auto x:l)  
    cout<<x<<" ";
```

Output:

5 10

When to use?

Rarely.

Use when:

Frequent insert/delete in middle

Interview importance:



2. MULTIMAP

Map that allows duplicate keys.

Header:

```
#include <map>
```

Declaration:

```
multimap<int,string> mp;
```

Example:

```
mp.insert({1,"A"});  
mp.insert({1,"B"});  
mp.insert({1,"C"});
```

Stored:

```
1 -> A  
1 -> B
```

```
1 -> C
```

Difference:

Map	Multimap
Unique key	Duplicate key allowed

Important:

```
mp[key]
```

✗ Not supported.

Use:

```
insert()  
find()  
equal_range()
```

Interview importance:



3. CUSTOM COMPARATOR

Used in:

```
sort()  
priority_queue()  
set()  
map()
```

Sort Descending:

```
sort(v.begin(), v.end(), greater<int>());
```

Custom Sort Pair by Second Value:

```
bool cmp(pair<int,int> a,
        pair<int,int> b)
{
    return a.second < b.second;
}

sort(v.begin(), v.end(), cmp);
```

Example:

Input:

```
(1,5)
(2,1)
(3,3)
```

Output:

```
(2,1)
(3,3)
(1,5)
```

Interview importance:



Actually very important.

4. next_permutation()

Generates next lexicographical permutation.

Header:

```
#include <algorithm>
```

Example:

```
vector<int> v={1,2,3};

next_permutation(
    v.begin(),
    v.end()
);
```

Output:

```
1 3 2
```

Generate all permutations:

```
sort(v.begin(),v.end());

do
{
    for(auto x:v)
        cout<<x<<" ";

    cout<<endl;

}
while(
next_permutation(
v.begin(),
v.end()
));
```

Output:

```
123
132
213
231
312
321
```

Interview importance:



5. __builtin_popcount()

Counts set bits.

Example:

```
__builtin_popcount(13);
```

Binary:

```
1101
```

Output:

```
3
```

64-bit:

```
__builtin_popcountll(x);
```

Examples:

```
__builtin_popcount(7);
```

```
111
```

Output:

```
3
```

Interview importance:



Very common in Bit Manipulation.

6. Important STL Algorithms

Header:

```
#include <algorithm>
```

Already know:

```
sort()
reverse()
binary_search()
lower_bound()
upper_bound()
min_element()
max_element()
count()
find()
```

Additional useful ones:

all_of()

```
all_of(
v.begin(),
v.end(),
[](int x)
{
    return x>0;
}
);
```

All positive?

any_of()

```
any_of(...)
```

At least one satisfies.

none_of()

```
none_of(...)
```

None satisfy.

gcd()

```
#include <numeric>

gcd(12,18);
```

Output:

```
6
```

iota()

Fill sequence.

```
vector<int> v(5);

iota(
v.begin(),
v.end(),
1
);
```

Output:

```
1 2 3 4 5
```

unique()

Remove adjacent duplicates.

```
v.erase(  
    unique(v.begin(),v.end()),  
    v.end()  
);
```

Input:

```
1 1 2 2 3
```

Output:

```
1 2 3
```

7. LAMBDA FUNCTIONS

Modern C++ feature.

Instead of:

```
bool cmp(int a,int b)  
{  
    return a>b;  
}
```

Use:

```
auto cmp =  
[](int a,int b)  
{  
    return a>b;  
};
```

Sorting:

```
sort(  
v.begin(),  
v.end(),  
[](int a,int b)  
{  
    return a>b;  
}  
);
```

Descending sort.

Interview importance:



Very useful.

8. CUSTOM HASH

Needed when:

```
unordered_map<pair<int,int>,int>
```

doesn't work.

Example:

```
struct custom_hash  
{  
    size_t operator()(  
        const pair<int,int>& p  
    ) const  
    {  
        return p.first*31  
            + p.second;  
    }  
};
```

Use:

```
unordered_map<  
pair<int,int>,  

```

```
int,  
custom_hash  
> mp;
```

Interview importance:



Rare.

9. PBDS (Policy Based Data Structure)

GNU Extension.

Header:

```
#include <ext/pb_ds/assoc_container.hpp>
```

Ordered Set:

```
tree<  
int,  
null_type,  
less<int>,  
rb_tree_tag,  
tree_order_statistics_node_update  
> os;
```

Functions:

`order_of_key(x)`

Count elements < x

```
os.order_of_key(5);
```

`find_by_order(k)`

k-th smallest element

```
*os.find_by_order(2);
```

Interview importance:



Competitive Programming only.

Rare in interviews.

What You Should Actually Master

For Interviews:

```
Vector  
Pair  
Sort  
Binary Search  
  
Stack  
Queue  
  
Set  
Map  
  
Unordered Set  
Unordered Map  
  
Priority Queue  
  
lower_bound  
upper_bound  
  
Custom Comparator
```

Master these 100%.

Final Interview Importance Ranking

Topic	Importance
Vector	★★★★★
Map	★★★★★
Unordered Map	★★★★★
Set	★★★★★
Binary Search	★★★★★
lower_bound / upper_bound	★★★★★
Priority Queue	★★★★★
Sort + Comparator	★★★★★
Stack	★★★★
Queue	★★★★
Multiset	★★★★
Lambda	★★★
next_permutation	★★★
__builtin_popcount	★★★
List	★★
Custom Hash	★★
Multimap	★
PBDS	★

Final STL Checklist

Before moving to DSA problems, make sure you can write from memory:

```
vector
pair
sort
reverse
binary_search

stack
queue
deque

set
unordered_set

map
```

`unordered_map`

`priority_queue`

`lower_bound`

`upper_bound`

`multiset`

`custom comparator`

If you can use all of these without looking up syntax, you're already at an **interview-ready STL level**.